Practical - 10

Practical Title: Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Problem Definition: Develop secure login and logout functionality using PHP sessions. Verify password using password_verify(), regenerate session ID after login, restrict dashboard access, implement role-based redirection, session timeout, and optional secure remember-me token.

Theory

1. Are sessions started and destroyed securely?

Answer: In PHP, session_start() is used to start a session or resume an existing session. After a user successfully logs in, session variables can store information such as the user's ID and role. For better security, session_regenerate_id(true) should be used after login to prevent session fixation attacks. When the user logs out, session_unset() removes session variables and session_destroy() destroys the session

Reference Link:
https://www.w3schools.com/php/php_sessions.asp

2. Is password_verify() used correctly?

Answer: Yes. The password_verify() function checks whether a password entered by the user matches the hashed password stored in the database. During registration, password_hash() should be used to store the password securely. During login, the entered password is checked against the stored hash using password_verify(). This avoids storing plain-text passwords in the database.

Reference Link:

3. Is unauthorized dashboard access blocked?

Answer: Yes. The dashboard should check whether the user has an active, valid login session before displaying protected content. If the session is missing, the user should be redirected to the login page. Checking authentication on the dashboard's server-side PHP code prevents users from accessing it simply by entering the URL directly in the browser.

Reference Link:
https://www.w3schools.com/php/php_sessions.asp

4. Is session timeout implemented?

Answer: Yes. Session timeout can be implemented by storing the time of the user's last activity in a session variable. Whenever the user visits a protected page, PHP checks the elapsed time. If the user remains inactive beyond the specified limit, such as 15 minutes, the session is destroyed and the user is redirected to the login page.

Reference Link:
https://www.w3schools.com/php/php_sessions.asp
https://www.w3schools.com/php/php_date.asp

5. Are admin and student users redirected to appropriate dashboards?

Answer: Yes. Role-based access control (RBAC) allows the application to provide different access permissions to different types of users. After successful login, PHP checks the user's role stored in the database and redirects the user to the appropriate dashboard. For example, an administrator is redirected to admin_dashboard.php, while a student is redirected to student_dashboard.php. Each dashboard must also verify the user's role to prevent unauthorized access.

Reference Link:
https://www.w3schools.com/php/php_sessions.asp
https://www.w3schools.com/php/php_if_else.asp

A. Implementation/ Output/Result

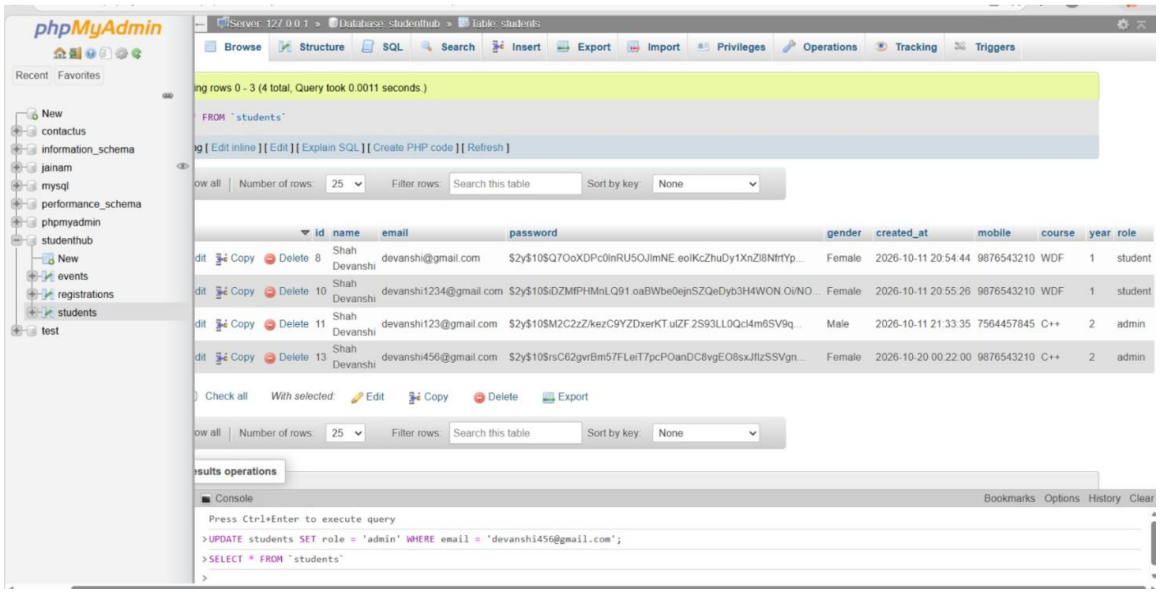
session.php

```
B.
C. <?php
D.
E. // Configure the session cookie before starting the session.
F. $isHttps = !empty($_SERVER["HTTPS"])
G.     && $_SERVER["HTTPS"] !== "off";
H.
I. session_set_cookie_params([
J.     "lifetime" => 0,
K.     "path" => "/",
L.     "secure" => $isHttps,
M.     "httponly" => true,
N.     "samesite" => "Lax"
O. ]);
P.
```

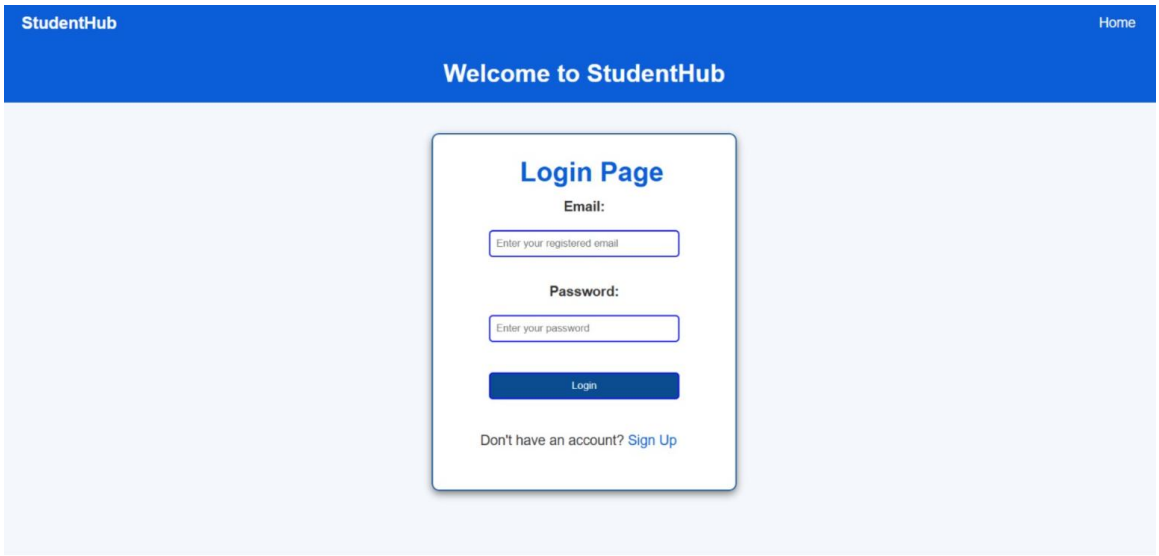
```
Q. if (session_status() !== PHP_SESSION_ACTIVE) {
R.     session_start();
S. }
T.
U. // Set the inactivity timeout to 15 minutes.
V. $timeout = 10;
W.
X. if (isset($_SESSION["last_activity"])) {
Y.
Z.     if (time() - $_SESSION["last_activity"] > $timeout) {
AA.
BB.         $_SESSION = [];
CC.
DD.         if (ini_get("session.use_cookies")) {
EE.             $params = session_get_cookie_params();
FF.
GG.             setcookie(
HH.                 session_name(),
II.                 "",
JJ.                 time() - 42000,
KK.                 $params["path"],
LL.                 $params["domain"],
MM.                 $params["secure"],
NN.                 $params["httponly"]
OO.             );
PP.         }
QQ.
RR.         session_destroy();
SS.
TT.         header("Location: login.php?timeout=1");
UU.         exit;
VV.     }
WW.}
XX.
YY.// Update the activity time.
ZZ.if (isset($_SESSION["user_id"])) {
AAA.       $_SESSION["last_activity"] = time();
BBB.   }
CCC.
DDD.   ?>

EEE.
```

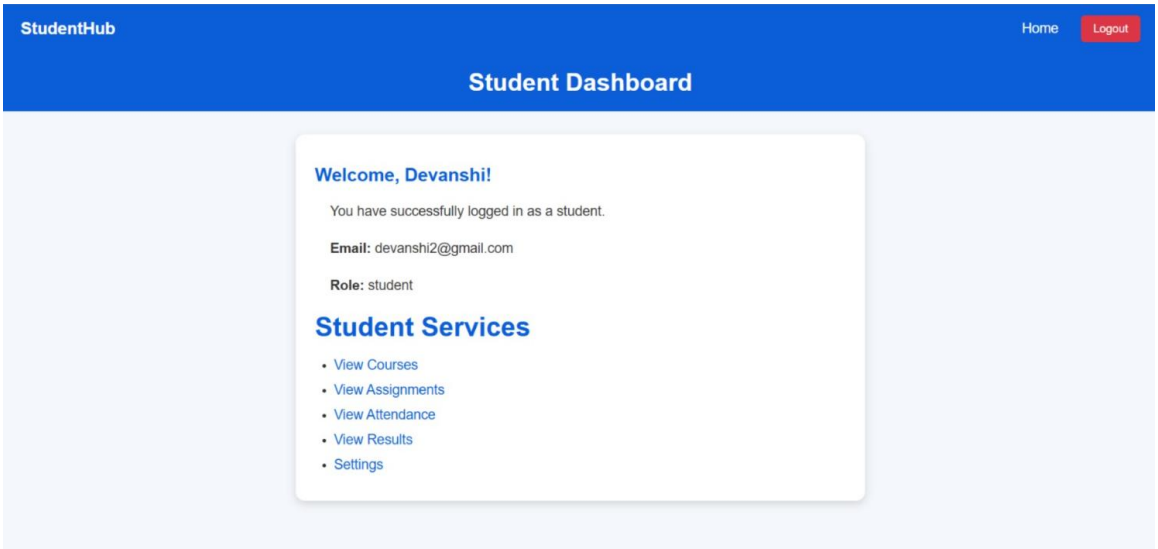
B. Evidence of Work (Screenshots)



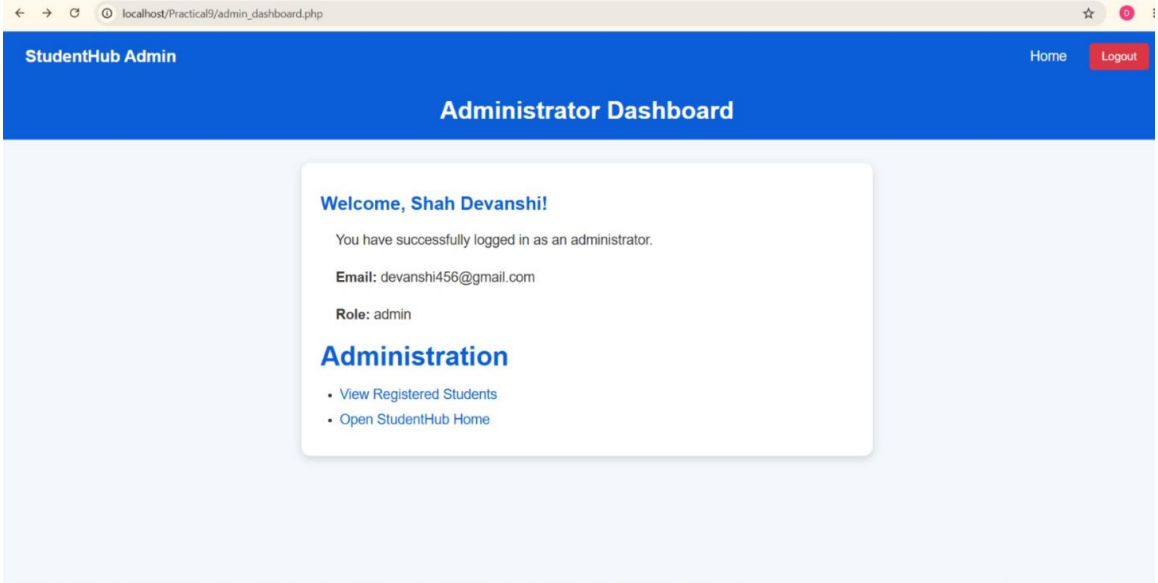
Description: This screenshot shows the students table of the studenthub database in phpMyAdmin. The table contains registered users along with their email addresses, securely hashed passwords, gender, mobile number, course, year, and role. The records demonstrate that both student and administrator roles are stored in the database. The password values are displayed as hashes rather than plain-text passwords, which supports the secure authentication requirement of Practical 10. The SQL console also shows an update query used to assign the admin role to a particular user, demonstrating role-based access control.



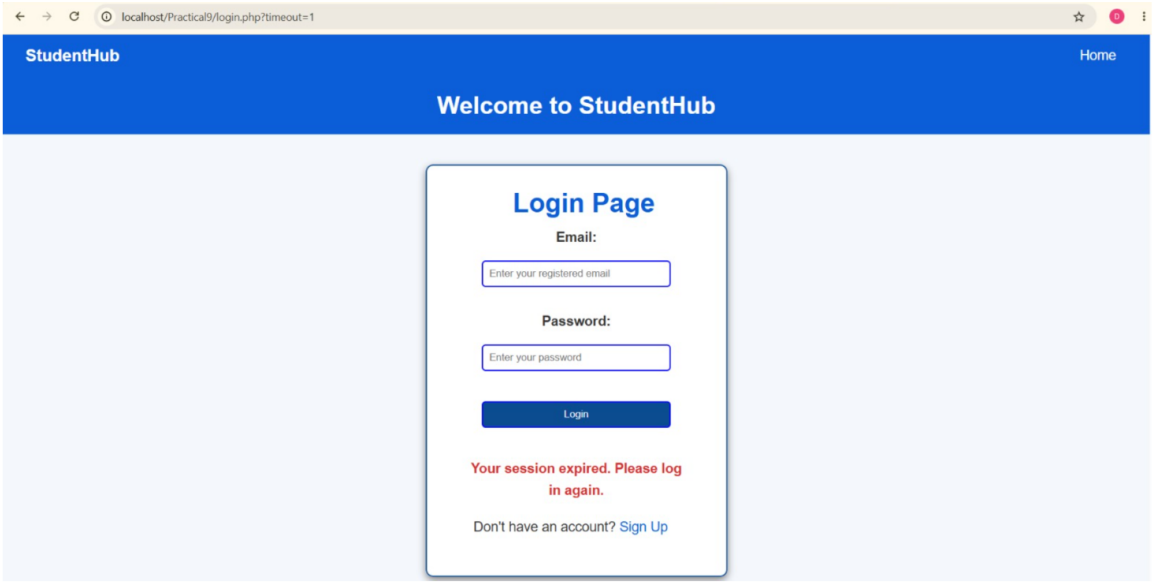
Description: This screenshot displays the StudentHub login page, where users are required to enter their registered email address and password. The page provides a Login button and a Sign Up option for new users. This demonstrates the authentication interface implemented for the practical. During login, the entered password can be checked against the hashed password stored in the database using password_verify(), ensuring that passwords are not compared or stored in plain text.



Description: This screenshot shows the Student Dashboard after a successful login by a student account. The dashboard displays a welcome message, the student's email address, and the role as student. It also provides student-specific services such as viewing courses, assignments, attendance, results, and settings. The presence of a Logout button demonstrates session-based authentication, while the student-specific dashboard confirms that role-based redirection is functioning correctly.



Description: This screenshot shows the Administrator Dashboard after successful authentication using an account with the admin role. The page displays the administrator's name, email address, and role, along with administrative options such as View Registered Students and Open StudentHub Home. This confirms the implementation of role-based access control, where an administrator is redirected to a separate dashboard and receives administrative privileges that are not available on the student dashboard.



Description: This screenshot demonstrates the session timeout functionality. After the session expires, the user is redirected back to the login page with the message “Your

session expired. Please log in again." This confirms that the application does not allow an expired authenticated session to continue accessing protected pages. The timeout mechanism improves security by automatically invalidating inactive sessions and requiring the user to authenticate again.

C. GitHub Repository Link:

https://github.com/devanshirshah24-pixel/WDF-Practicals/tree/main/Practical10

D. Challenges Faced:

- Session management: Understanding how to start, maintain, regenerate, and destroy PHP sessions securely was challenging.
- Password verification: Correctly verifying the entered password against the hashed password stored in the database required careful implementation.
- Unauthorized access prevention: Ensuring that users cannot open protected dashboard pages without logging in was an important challenge.
- Role-based redirection: Implementing separate dashboards for admin and student users required checking the user's role correctly.
- Session timeout: Checking user inactivity and automatically logging out inactive users required proper time calculations.
- Cookie and token security: Understanding how to handle remember-me cookies securely without exposing sensitive authentication information was challenging.

E. Learning Outcomes:

- Implement secure login and logout functionality using PHP sessions.
- Use `password_hash()` and `password_verify()` for secure password storage and verification.
- Regenerate session IDs after successful login to improve session security.
- Restrict dashboard access to authenticated users and prevent unauthorized access.

- Implement role-based access control for admin and student users.
- Configure session timeout to automatically log out inactive users.
- Understand the purpose of cookies and the security requirements of a remember-me token.
- Test successful login, failed login, logout, session timeout, and unauthorized dashboard access.

Faculty Remarks: (Keep this blank for faculty)